

Worksheet

Computer Network - Week 7 - Process Management

Name	Farrel Tsaqif Anindyo
Class	CSA

Activity 1

Procedure

Read the procedure on the activity file.

Answers

1. What was the initial state of the sleep process? Explain what this state means.

```
(farrelta@ROG)-[~]  
$ sleep 300 &  
[1] 2379  
  
(farrelta@ROG)-[~]  
$ ps -u | grep sleep  
farrelta  2379  0.0  0.0  5544  1760 pts/0    S   13:52   0:00  sleep 300  
farrelta  2381  0.0  0.0  6600  2080 pts/0    S+  13:52   0:00  grep --color=auto  sleep
```

It was in the waiting state that means it is waiting for a specific event to occur, in this case, the expiration of the specified time period

2. What happened to the process state after sending **SIGSTOP**? Describe the transition.

```
(farrelta@ROG)-[~]  
$ kill -STOP 2379  
  
[1]+  Stopped                  sleep 300  
(farrelta@ROG)-[~]  
$ ps -u | grep sleep  
farrelta  2379  0.0  0.0  5544  1760 pts/0    T   13:52   0:00  sleep 300  
farrelta  2389  0.0  0.0  6600  2080 pts/0    S+  13:53   0:00  grep --color=auto  sleep
```

It was in a stopped state. In this state, the process is put on hold and is not responsive.

3. How did **SIGCONT** affect the process? What state did it return to?

```
(farrelta@ROG)~$ kill -CONT 2379

(farrelta@ROG)~$ ps -u | grep sleep
farrelta  2379  0.0  0.0  5544  1760 pts/0    S   13:52   0:00 sleep 300
farrelta  2391  0.0  0.0  6600  2080 pts/0    S+  13:53   0:00 grep --color=auto sleep
```

It resumed the process and returned the initial waiting state

4. Explain the difference between **SIGSTOP** and **SIGTERM** signals.

```
(farrelta@ROG)~$ kill -SIGTERM 2379

(farrelta@ROG)~$ ps -u | grep sleep
farrelta  2394  0.0  0.0  6600  2080 pts/0    S+  13:53   0:00 grep --color=auto sleep
[1]+  Terminated                  sleep 300
```

“SIGSTOP” just stop/resume the process while “SIGTERM” terminate/remove the process completely

5. What happens when you use **SIGKILL** instead? Explain the difference between **SIGKILL** and **SIGTERM**

```
(farrelta@ROG)~$ kill -SIGTERM 2379

(farrelta@ROG)~$ ps -u | grep sleep
farrelta  2394  0.0  0.0  6600  2080 pts/0    S+  13:53   0:00 grep --color=auto sleep
[1]+  Terminated                  sleep 300
```

“SIGTERM” terminates the process gracefully allowing the process to finish tasks, save data, and release resources while “SIGKILL” terminates the process forcefully without any cleanup

Activity 2

Procedure

Read the procedure on the activity file.

Answers

1. What is the difference between a foreground and background process?

```
(farrelta@ROG)~$ sleep 100
^Z
[1]+  Stopped                  sleep 100

(farrelta@ROG)~$ jobs
[1]+  Stopped                  sleep 100

(farrelta@ROG)~$ bg
[1]+ sleep 100 &

(farrelta@ROG)~$ jobs
[1]+  Running                  sleep 100 &
```

Foreground process runs interactively with the user so we can see the process running directly while background process run in the background so we can't see the process directly

2. What happened when you pressed CTRL+Z? What signal was sent?

It stop the process and put it in waiting state

3. Explain the purpose of the `jobs` command and what information it provides.

It lists the active process with the state in the current running terminal

Activity 3

Procedure

Read the procedure on the activity file.

Answers

1. What is a zombie process? When does it occur?

```
(farrelta@ROG)-[~]
$ ./zombie_demo &
[1] 2805

(farrelta@ROG)-[~]
$ Parent Process (PID: 2805) is starting...
Parent Process: Created child with PID 2806
Parent Process: Sleeping for 20 seconds...
Parent Process: Check process table NOW to see zombie!
Child Process (PID: 2806) is running...
Child Process: My parent is PID 2805
Child Process: Exiting now...
Parent Process: Finished sleeping. Zombie should be gone now.
^C

[1]+  Done                  ./zombie_demo
```

It is the process that has finished execution, but its parent hasn't read its exit status yet. It occurs when the process has completed the execution but still exists in the process table because the parent hasn't yet collected the exit process.

2. In your screenshot, what state is the child process in (check STAT column)? What does <defunct> mean?

```

(farrelta@ROG)-[~]
$ ps aux | grep zombie_demo
farrelta    2805  0.0  0.0  2560  1440 pts/0    S   14:39   0:00 ./zombie_demo
farrelta    2806  0.0  0.0      0      0 pts/0    Z   14:39   0:00 [zombie_demo] <defunct>
farrelta    2810  0.0  0.0  6604  2080 pts/2    S+  14:40   0:00 grep --color=auto zombie_demo

(farrelta@ROG)-[~]
$ ps -el | grep zombie_demo
0 S  1000    2805    2362  0  80    0 -    640 hrtime pts/0    00:00:00 zombie_demo
1 Z  1000    2806    2805  0  80    0 -      0 -      pts/0    00:00:00 zombie_demo

(farrelta@ROG)-[~]
$ ps -el | grep zombie_demo

(farrelta@ROG)-[~]
$ ps aux | grep zombie_demo
farrelta    2819  0.0  0.0  6604  2080 pts/2    S+  14:40   0:00 grep --color=auto zombie_demo

```

It was in "Z"(zombie) state, <defunct> is another name for zombie state

3. Why did the zombie process disappear after the parent process finished?

its children are sent a SIGHUP signal. SIGHUP is a catchable signal, but if the child doesn't catch it, the default action is to kill the child process

Activity 4

Procedure

Read the procedure on the activity file.

Answers

1. What is the purpose of the `wait()` system call?

The `wait()` system call's purpose is to suspend the parent process until one of its child processes terminates

2. Compare `zombie_demo.c` with `no_zombie.c`. What is the main difference that prevents zombies?

```
(farrelta@ROG)-[~]
$ touch no_zombie.c && nano no_zombie.c

(farrelta@ROG)-[~]
$ gcc -o no_zombie no_zombie.c

(farrelta@ROG)-[~]
$ ./no_zombie &
[1] 2874

(farrelta@ROG)-[~]
$ Parent Process (PID: 2874) is starting...
Parent Process: Created child with PID 2875
Parent Process: Waiting for child to complete...
Child Process (PID: 2875) is running...
Child Process: My parent is PID 2874
Child Process: Doing some work...
Child Process: Work completed. Exiting with status 0...
Parent Process: Child exited normally with status: 0
Parent Process: Child has been reaped. No zombie created!
Parent Process: Sleeping for 10 more seconds...
Parent Process: Exiting now.
```

Because the parent is waiting for the child process to complete after that the parent finish its process

3. What information does the status parameter in `wait(&status)` contain? Explain `WIFEXITED()` and `WEXITSTATUS()`

```
(farrelta@ROG)-[~]
$ ps aux | grep no_zombie
farrelta  2874  0.0  0.0  2560  1440 pts/0    S   14:50   0:00 ./no_zombie
farrelta  2877  0.0  0.0  6604  1920 pts/2    S+  14:50   0:00 grep --color=auto no_zombie

(farrelta@ROG)-[~]
$ ps -el | grep no_zombie
0 S  1000  2874  2362  0  80  0 -  640 hrtme pts/0  00:00:00 no_zombie
```

It contains the status process of the child,
`WIFEXITED()` is used to check if the child process terminated normally
`WEXITSTATUS()` is used to get the specific exit status code from a child process that terminated normally